APPLIED BUSINESS ANALYTICS

All 12 Practical Question Bank Answers

T.Y.B.Sc. (I.T.) Semester V | Minor – Data Science | Practical Examination 2026

Easy English notes with theory, formulas, Python programs, examples, and interpretations.

Note: The examples use illustrative sample data. The actual Sales.csv dataset was not provided in the question bank.

Contents

1. Exploratory Data Analysis (EDA)
2. Central Limit Theorem
3. Descriptive Statistics using Sales.csv
4. Confidence Intervals
5. Histograms and Box Plots
6. Bootstrapping
7. Correlation Analysis
8. Hypothesis Testing
9. Probability Simulation
10. Random Probability Distributions
11. Decision Tree for Business Decisions
12. Expected Value and Optimal Decision

Q1. Collect Data from a Real-Life Business Scenario and Perform EDA

Aim

To collect business data and perform Exploratory Data Analysis to understand the dataset and identify useful insights.

Theory

Exploratory Data Analysis (EDA) means examining data before using advanced statistical or machine-learning methods. It includes checking the dataset structure, missing values, duplicates, summary statistics, patterns, and unusual values.

Example

Retail shop sample: Monday Pen 10 units at ■10 = ■100; Tuesday Notebook 5 units at ■50 = ■250; Wednesday Pen 15 units at ■10 = ■150; Thursday Bag 2 units at ■500 = ■1,000; Friday Notebook 8 units at ■50 = ■400. Total sales = ■1,900.

Python Program

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Day": ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"],
    "Product": ["Pen", "Notebook", "Pen", "Bag", "Notebook"],
    "Units_Sold": [10, 5, 15, 2, 8],
    "Price": [10, 50, 10, 500, 50],
    "Sales": [100, 250, 150, 1000, 400]
}
df = pd.DataFrame(data)
print(df)
print(df.describe())
print(df.isnull().sum())
print("Total sales:", df["Sales"].sum())

df["Sales"].plot(kind="bar", title="Daily Sales")
plt.xlabel("Day")
plt.ylabel("Sales (Rs.)")
plt.tight_layout()
plt.show()
```

Interpretation

Total sales are ■1,900. Thursday has the highest sales (■1,000) and Monday has the lowest (■100). The shop can use this information to plan stock and promotions.

Conclusion

EDA helps businesses understand their data, discover patterns, and make better decisions.

Q2. Use the Central Limit Theorem to Analyze a Sampling Distribution

Aim

To understand the Central Limit Theorem and use sample means to estimate population characteristics.

Theory

The Central Limit Theorem says that, under suitable conditions, the sampling distribution of the sample mean becomes approximately normal as sample size increases. The mean of sample means is μ and the standard error is $SE = \sigma/\sqrt{n}$.

Example

Suppose mean daily sales μ = ■5,000, standard deviation σ = ■1,000, and sample size n = 100. Then $SE = 1000/\sqrt{100}$ = ■100. The sampling distribution is centred around ■5,000.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt
```

```
np.random.seed(42)
population = np.random.exponential(scale=5000, size=100000)
sample_means = []

for i in range(1000):
    sample = np.random.choice(population, size=100, replace=True)
    sample_means.append(np.mean(sample))

print("Population mean:", np.mean(population))
print("Mean of sample means:", np.mean(sample_means))
print("Standard error:", np.std(sample_means))

plt.hist(sample_means, bins=30, edgecolor="black")
plt.title("Sampling Distribution of Sample Mean")
plt.xlabel("Sample mean")
plt.ylabel("Frequency")
plt.show()
```

Interpretation

The histogram of sample means should look more bell-shaped than the original skewed population. The mean of sample means should be close to the population mean; exact values vary due to random sampling.

Conclusion

CLT supports confidence intervals, hypothesis tests, and estimation from samples.

Q3. Calculate Descriptive Statistics using Sales.csv

Aim

To calculate mean, median, mode, variance, and other descriptive statistics for a variable in Sales.csv.

Theory

Descriptive statistics summarize data. Mean is the arithmetic average; median is the middle value; mode is the most frequent value; variance measures spread around the mean; standard deviation is the square root of variance. Quartiles describe the spread of the middle part of data.

Example

Example sales values: ■100, ■200, ■200, ■300, ■700. Mean = ■300; median = ■200; mode = ■200. The values show that one high sale increases the mean.

Python Program

```
import pandas as pd

df = pd.read_csv("Sales.csv")
print(df.head())
print(df.columns)

# Change "Sales" to the exact numeric column name in your file
x = df["Sales"].dropna()
print("Mean:", x.mean())
print("Median:", x.median())
print("Mode:", x.mode().tolist())
print("Variance:", x.var())
print("Standard deviation:", x.std())
print("Minimum:", x.min())
print("Maximum:", x.max())
print("Quartiles:")
print(x.quantile([0.25, 0.50, 0.75]))
```

Interpretation

Mean shows average sales, median gives the middle sale, and mode gives the most common sale. A large difference between mean and median may indicate skewed data or unusually high/low values.

Conclusion

Descriptive statistics provide a quick summary of sales performance. Use the actual column name found in Sales.csv.

Q4. Calculate Confidence Intervals for Population Means or Proportions

Aim

To estimate a population mean or proportion from sample data and explain the estimate in a business context.

Theory

A confidence interval is a range of plausible values for a population parameter. For a mean with known population standard deviation, a 95% interval is x■ ± 1.96(σ/√n). For a large-sample proportion, an approximate 95% interval is p■ ± 1.96√[p■(1−p■)/n].

Example

Suppose average customer spending in a sample is ■500, population standard deviation is ■100, and n = 100. Standard error = 100/10 = ■10. The 95% interval is 500 ± 1.96(10), or ■480.40 to ■519.60.

Python Program

```
import math

mean = 500
sigma = 100
n = 100
z = 1.96

se = sigma / math.sqrt(n)
lower = mean - z * se
upper = mean + z * se

print("Standard error:", se)
print("95% confidence interval:", lower, "to", upper)

# Proportion example: 60 satisfied customers out of 100
p = 60 / 100
se_p = math.sqrt(p * (1 - p) / 100)
print("Proportion CI:",
      p - 1.96 * se_p, "to", p + 1.96 * se_p)
```

Interpretation

The estimated mean spending is ■500, with a 95% confidence interval of about ■480.40–■519.60. In repeated sampling, the method would produce intervals containing the true population mean about 95% of the time under its assumptions.

Conclusion

Confidence intervals communicate both an estimate and its uncertainty. For small samples with unknown population standard deviation, a t-interval is generally used.

Q5. Create Histograms and Box Plots

Aim

To visualize the distribution of a variable and identify its shape, spread, and possible outliers.

Theory

A histogram groups numerical values into bins and shows their frequencies. A box plot displays the median, lower and upper quartiles, and possible outliers. These graphs help detect skewness, spread, clusters, and unusual observations.

Example

Example daily sales: 100, 120, 130, 150, 160, 180, 200, 220, 250, 900. The value 900 is much larger than the others and may be an unusual high sale.

Python Program

```
import matplotlib.pyplot as plt

sales = [100, 120, 130, 150, 160, 180, 200, 220, 250, 900]
```

```
plt.hist(sales, bins=5, edgecolor="black")
plt.title("Histogram of Daily Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()

plt.boxplot(sales)
plt.title("Box Plot of Daily Sales")
plt.ylabel("Sales")
plt.show()
```

Interpretation

The histogram shows how often sales fall into different ranges. The box plot can flag 900 as a possible outlier depending on the quartiles. Check whether an unusual value is an error or a genuine large sale before removing it.

Conclusion

Histograms and box plots make distributions easier to understand and help guide further analysis.

Q6. Apply Bootstrapping to Estimate Confidence Intervals

Aim

To estimate a confidence interval for a statistic using repeated resampling from observed data.

Theory

Bootstrapping is a computer-based method that repeatedly samples observations from the original sample with replacement. A statistic is calculated for every bootstrap sample. Percentiles of the bootstrap statistics can be used as a confidence interval. It is useful when a simple theoretical formula is difficult to apply.

Example

Sample customer spending values are ■200, ■250, ■300, ■350, and ■900. Repeatedly resampling these five values allows us to estimate uncertainty around the median without assuming a normal population.

Python Program

```
import numpy as np

np.random.seed(42)
data = np.array([200, 250, 300, 350, 900])
boot_medians = []

for i in range(10000):
    sample = np.random.choice(data, size=len(data), replace=True)
    boot_medians.append(np.median(sample))

lower, upper = np.percentile(boot_medians, [2.5, 97.5])
print("Sample median:", np.median(data))
print("Bootstrap 95% CI for median:", lower, "to", upper)
```

Interpretation

The output gives a bootstrap interval for the median. With a very small original sample, the interval may be unstable; more representative observations usually improve the analysis.

Conclusion

Bootstrapping estimates statistical uncertainty by resampling the observed data. It does not automatically fix a biased or unrepresentative sample.

Q7. Perform Correlation Analysis on Multiple Variables

Aim

To measure the strength and direction of relationships between pairs of numerical variables.

Theory

Correlation measures how two variables move together. Pearson's correlation coefficient r ranges from −1 to +1. A positive value indicates a positive linear relationship, a negative value indicates a negative linear relationship, and a value near zero indicates little linear relationship. Correlation does not prove causation.

Example

Example: a shop records advertising spend and sales. If higher advertising spend tends to occur with higher sales, the correlation may be positive. Other factors, such as season or discounts, may also affect sales.

Python Program

```
import pandas as pd

data = {
    "Advertising": [10, 20, 30, 40, 50],
    "Sales": [100, 130, 170, 200, 240],
    "Discount": [5, 5, 10, 10, 15]
}
df = pd.DataFrame(data)

print(df.corr(numeric_only=True))

import matplotlib.pyplot as plt
plt.scatter(df["Advertising"], df["Sales"])
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.title("Advertising and Sales")
plt.show()
```

Interpretation

A positive correlation between advertising and sales suggests that they increase together in this sample. It does not prove that advertising alone caused the increase.

Conclusion

Correlation analysis helps businesses identify relationships and generate questions for further investigation.

Q8. Formulate Hypotheses and Conduct a Hypothesis Test

Aim

To test a business claim using sample data and interpret the statistical result.

Theory

A null hypothesis (H■) usually represents no difference or a stated baseline. The alternative hypothesis (H■) represents the effect or difference being investigated. A significance level such as 0.05 is selected. If the p-value is less than the significance level, reject H■; otherwise, fail to reject H■. This does not prove H■ is true.

Example

Business question: has a new advertising campaign changed average daily sales? H■: mean sales before and after are equal. H■: mean sales differ. If the same stores are measured before and after, a paired test may be suitable.

Python Program

```
import numpy as np
from scipy import stats

before = np.array([100, 110, 105, 98, 120, 115])
after  = np.array([115, 120, 112, 108, 130, 125])

t_stat, p_value = stats.ttest_rel(after, before)
alpha = 0.05

print("t statistic:", t_stat)
print("p-value:", p_value)

if p_value < alpha:
    print("Reject H0: evidence of a change.")
else:
    print("Fail to reject H0: insufficient evidence of a change.")
```

Interpretation

The program compares paired observations. A p-value below 0.05 indicates evidence of a difference under the test assumptions. The conclusion should also consider sample size, effect size, and business importance.

Conclusion

Hypothesis testing provides a structured method for evaluating business claims using data.

Q9. Simulate a Probability Experiment

Aim

To simulate a random experiment with programming and estimate probabilities from observed outcomes.

Theory

Probability is a number from 0 to 1 describing the likelihood of an event. In simulation, a random experiment is repeated many times. Estimated probability equals the number of times the event occurs divided by the total number of trials.

Example

Example: when a fair die is rolled, the theoretical probability of getting a six is 1/6, or about 0.1667.

Python Program

```
import random

random.seed(42)
trials = 10000
count_six = 0

for i in range(trials):
    result = random.randint(1, 6)
    if result == 6:
        count_six += 1

estimated_probability = count_six / trials
print("Trials:", trials)
print("Number of sixes:", count_six)
print("Estimated probability:", estimated_probability)
print("Theoretical probability:", 1/6)
```

Interpretation

The simulated probability should usually be near 0.1667 for a large number of trials, but it may not equal the theoretical probability exactly.

Conclusion

Simulation helps understand probability and model uncertain events in business and everyday life.

Q10. Generate Random Numbers from Probability Distributions

Aim

To generate random data from normal, uniform, and exponential distributions and examine their properties.

Theory

A normal distribution is bell-shaped and described by mean and standard deviation. A uniform distribution gives equal density over a specified interval. An exponential distribution is non-negative and often models waiting times between events in a constant-rate process.

Example

Examples: normal data may represent measurement errors; uniform data can represent a random value between 0 and 1; exponential data can model customer waiting times under suitable assumptions.

Python Program

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
```

```
normal_data = np.random.normal(loc=50, scale=10, size=1000)
uniform_data = np.random.uniform(low=0, high=1, size=1000)
exponential_data = np.random.exponential(scale=5, size=1000)

print("Normal mean:", np.mean(normal_data))
print("Uniform mean:", np.mean(uniform_data))
print("Exponential mean:", np.mean(exponential_data))

plt.hist(normal_data, bins=30, edgecolor="black")
plt.title("Normal Distribution")
plt.show()

plt.hist(uniform_data, bins=30, edgecolor="black")
plt.title("Uniform Distribution")
plt.show()

plt.hist(exponential_data, bins=30, edgecolor="black")
plt.title("Exponential Distribution")
plt.show()
```

Interpretation

Normal data should cluster around 50; uniform data should spread across 0 to 1; exponential data should be right-skewed. Sample statistics vary from the theoretical values.

Conclusion

Random distributions are useful for simulations, risk analysis, queue models, and understanding uncertainty.

Q11. Develop a Decision Tree Model for Business Decisions

Aim

To build a decision tree that helps choose a business action based on features and historical outcomes.

Theory

A decision tree is a supervised learning method that splits data into groups using feature-based rules. In classification, it predicts categories; in regression, it predicts numerical values. The tree is easy to interpret but may overfit if it is too deep.

Example

Example: a business may predict whether a customer will buy a product using age, income, and previous purchases. A tree learns decision rules from labelled historical data.

Python Program

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

data = pd.DataFrame({
    "Income": [20, 25, 30, 45, 50, 60, 65, 70],
    "PreviousPurchase": [0, 0, 1, 0, 1, 1, 1, 1],
    "Buy": ["No", "No", "No", "Yes", "Yes", "Yes", "Yes", "Yes"]
})

X = data[["Income", "PreviousPurchase"]]
y = data["Buy"]

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

print(export_text(model, feature_names=list(X.columns)))
print("Prediction:", model.predict([[55, 1]])[0])
```

Interpretation

The printed rules show how the tree separates customers into groups. The prediction for a new customer is based on the learned rules. A real business should evaluate the model on separate test data before using it.

Conclusion

A decision tree can support understandable predictions and business decisions, but its performance must be tested on unseen data.

Q12. Apply Expected Value to Select the Optimal Decision

Aim

To calculate expected monetary value for decision alternatives and select the option with the highest expected value.

Theory

Expected value (EV) is the sum of each possible outcome multiplied by its probability. For a decision with outcomes x■, x■, …, EV = Σ p■x■, where probabilities sum to 1. If the objective is to maximize profit, choose the alternative with the highest expected profit, while also considering risk and other business factors.

Example

Example: Option A has a 0.6 chance of earning ■10,000 and a 0.4 chance of earning ■2,000. EV(A) = 0.6(10,000) + 0.4(2,000) = ■6,800. Option B gives ■6,000 with certainty. Based on expected value alone, choose A because ■6,800 > ■6,000.

Python Program

```
options = {
    "Option A": [(0.6, 10000), (0.4, 2000)],
    "Option B": [(1.0, 6000)]
}

expected_values = {}

for name, outcomes in options.items():
    ev = sum(probability * payoff
             for probability, payoff in outcomes)
    expected_values[name] = ev
    print(name, "Expected Value = Rs.", ev)

best_option = max(expected_values, key=expected_values.get)
print("Best option by expected value:", best_option)
```

Interpretation

Option A has an expected value of ■6,800, while Option B has an expected value of ■6,000. Option A is preferred if maximizing expected monetary value is the only objective. A risk-averse business may consider the possibility of the lower outcome.

Conclusion

Expected value provides a clear numerical comparison of decisions under uncertainty. It should be combined with risk, costs, and business goals.

Quick Practical Exam Tips

1. Write the aim first, followed by theory, program, output or result, interpretation, and conclusion.
2. For Sales.csv, check the actual column names using print(df.columns) before selecting a variable.
3. Install required Python packages if missing: pandas, numpy, matplotlib, scipy, and scikit-learn.
4. Use the exact dataset and results provided by your teacher in the practical examination.
5. For hypothesis testing, state H0, H1, significance level, p-value, and conclusion.
6. For business interpretations, explain what the numerical result means for the business.